GOODS.NG

Buy. Sell. Trade with Confidence.

DOCUMENT 2 OF 3

Technical Architecture Document

How the system is structured, deployed, secured and operated, and the exact state of every external integration required for go-live.

22 sections · system context · data and ledger · payments · Flex · security · scalability · Appendix A/B/C (integration checklist, environment variables, glossary)

VERSION
1.0

DATE
8 November 2026

STATUS
Go-live planning baseline

MARKET
Nigeria · NGN

Confidential — internal product & engineering document

DOCUMENT CONTROL

About this document

This TAD describes the architecture behind the first release: the modular monolith, PostgreSQL source of truth, Redis cache, worker, provider interfaces, payments and escrow state machine, double-entry ledger, Goods Flex, search and the frontend. Appendix A is the integration go-live checklist.

Product	Goods.ng — multi-vendor marketplace for Nigeria
Owner	Product & Engineering, Goods.ng
Audience	Engineering, platform, security, QA and integration partners
Source of truth	Repository `docs/`, application code, migrations and automated tests as of 8 November 2026
Verification baseline	205 backend tests, 10 frontend unit tests, Playwright E2E, ruff/eslint/tsc clean, production build passing
Companion documents	PRD (priority authority) · TAD (implementation authority) · FRD (acceptance authority)

> **Integration caveat.** Production boot refuses mock financial providers. Protected Checkout and Goods Flex require licensed escrow and lending partner adapters before live use.

Contents

1. 1. Architecture principles
2. 2. System context
3. 3. Technology stack
4. 4. Deployment topology
5. 5. Backend architecture
6. 6. Data architecture
7. 7. API architecture
8. 8. Identity and access
9. 9. Payments, escrow and ledger
10. 10. Goods Flex
11. 11. Search and discovery
12. 12. Messaging and negotiation
13. 13. Frontend architecture
14. 14. Media pipeline
15. 15. Background processing
16. 16. Caching and performance
17. 17. Security architecture
18. 18. Observability and operations
19. 19. Integration architecture and go-live readiness
20. 20. Scalability plan
21. 21. Architecture decisions
22. 22. Technical risks

Appendix A. Integration go-live checklist

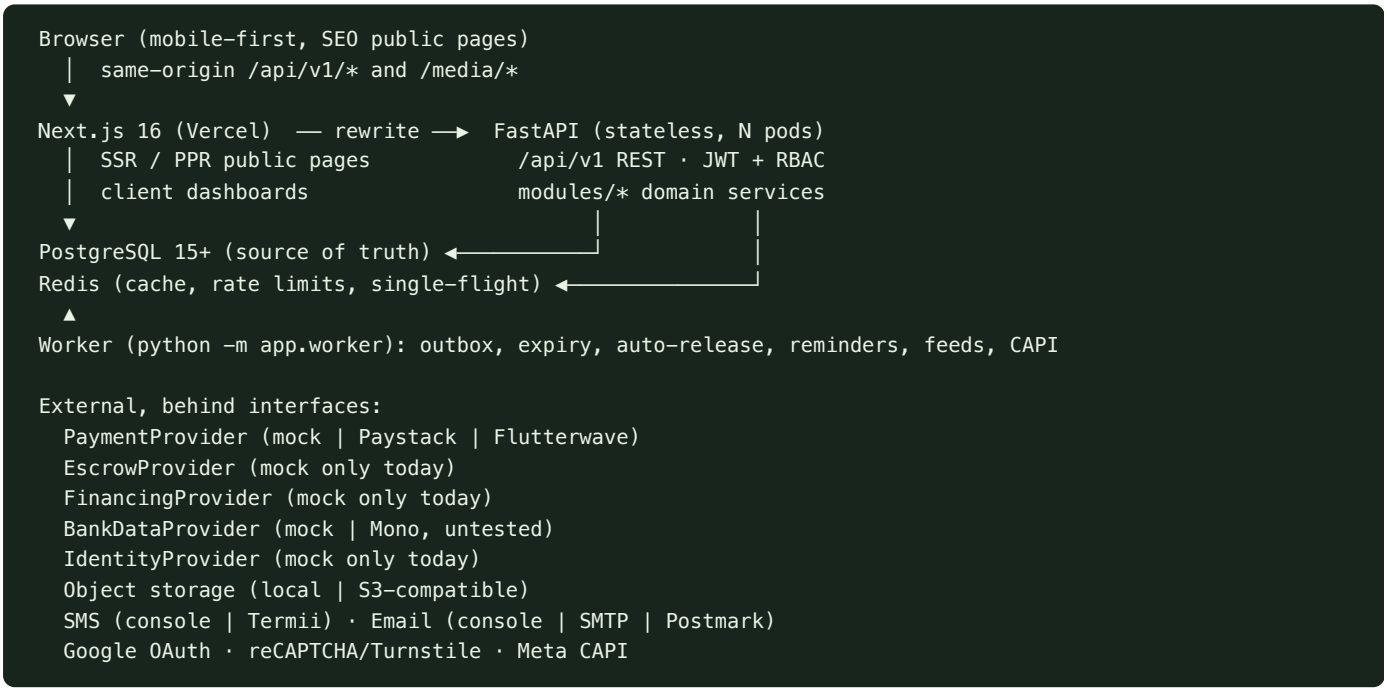
Appendix B. Environment variables

Appendix C. Glossary

1. Architecture principles

1. **Modular monolith first.** One deployable API with clear module boundaries and a single database. Scale the process horizontally before splitting services.
2. **Provider interfaces at every regulated or external boundary.** Payments, escrow, lending, open banking, identity, storage, SMS and email are behind Protocols so a partner can be replaced without touching domain logic.
3. **Verify before trust.** A payment is real only after a server-to-server verification with an exact amount and currency match. Webhooks trigger verification; redirects and screenshots never do.
4. **The database is the source of truth.** Financial invariants are enforced by constraints, row locks, unique keys and triggers, not only by application code.
5. **Idempotency everywhere.** Checkout, webhooks, settlement, refunds, promotions and BNPL payments can be retried safely.
6. **Same-origin API.** The browser talks only to the Next.js origin, which proxies `/api/v1` and `/media` to the backend. This keeps the refresh cookie first-party and removes CORS from the production path.
7. **Operate what you build.** Structured logs, request IDs, health checks, reconciliation and audit trails are part of the feature, not an afterthought.

2. System context



3. Technology stack

Layer	Technology	Notes
Frontend	Next.js 16.4 (App Router, Cache Components/PPR), React 19.3, TypeScript 5, Tailwind 4	System fonts, streaming SSR, client dashboards on a central API client.
Backend	Python 3.12, FastAPI, Pydantic 2, SQLAlchemy 2, Alembic	185 REST endpoints, 20 modules, OpenAPI only in dev/test.
Database	PostgreSQL 15+	54 tables, full-text search, partial unique indexes, append-only triggers, deferred balance check.
Cache / limits	Redis 5+	Required in production. In-memory fallback in development. Rate limits and single-flight locks.

Layer	Technology	Notes
Worker	Same image, `python -m app.worker`	Postgres advisory locks make multiple replicas safe.
Storage	Local (dev) or S3-compatible + CDN	Pillow decode, WebP re-encode, watermark, private KYC prefix.
Tests	pytest, Vitest, Playwright, ruff, eslint, tsc	205 backend, 5 unit, 6 E2E; CI runs migrations round-trip and build.
Deploy	Vercel (frontend) + container platform (API/worker) + managed Postgres/Redis	Dockerfiles and Compose exist; not yet deployed.

4. Deployment topology

Component	Production shape	Scaling / resilience
Frontend	Vercel, root `frontend/`, `pnpm build`; rewrites `/api/v1` and `/media` to the API.	CDN edge, static/PPR pages, per-request streaming for data.
API	Container image from `backend/Dockerfile`, ≥ 2 instances behind HTTPS.	Stateless; horizontal scale. Pool: (10 + 20) × workers × replicas.
Worker	Same image, separate command, ≥ 1 replica.	Advisory locks prevent duplicate job execution; scale horizontally.
Database	Managed Postgres 15+, TLS, PITR, private network; PgBouncer (transaction pooling) as replicas grow.	Read replicas for browse/search before write saturation; 15 s statement timeout.
Redis	Managed Redis, private network.	Cache, rate limits, single-flight. Boot fails without it in production.
Storage	S3-compatible bucket, versioning, CDN for public media, no public access for `private/`.	Signed URLs for KYC; lifecycle rules; cross-region replication for private data.
DNS	goods.ng, admin.goods.ng; SSL managed by the platform.	Admin host carries `X-Robots-Tag: noindex`; optional edge IP allow-list.

> **Connection budget**
> Each API process opens up to `DB_POOL_SIZE + DB_MAX_OVERFLOW` connections. Total = that × `WEB_CONCURRENCY` × replicas. Size this against `max_connections`, or put PgBouncer in front. This is the first thing to tune when adding pods.

5. Backend architecture

The API is a modular monolith. Each module owns its router and business logic; cross-module coupling goes through service functions and the payment purpose registry. The payments module never imports orders, subscriptions or BNPL directly.

Layer	Responsibility	Examples
`core/`	Config, DB session, security primitives, errors, JSON logging with request IDs, cache/rate limiter, auth/RBAC dependencies.	Argon2, JWT, Fernet, Redis, trusted proxy IP resolution.
`models/`	SQLAlchemy mappings and constraints.	Users, sellers, catalog, messaging, commerce, ledger, misc.
`integrations/`	External provider Protocols and adapters.	Payments, escrow, BNPL, bank data, identity, storage.

Layer	Responsibility	Examples
`modules/`	Domain services, routers, state machines, policies.	auth, users, sellers, categories, products, subscriptions, messaging, negotiation, payments, orders, promotions, bnpl, reviews, notifications, admin, staff, stats, recommendations, ads, marketing.
`worker.py`	Scheduled jobs with advisory locks.	Outbox, expiry, auto-release, subscription/promotion expiry, BNPL reminders, feed/CAPI, purge.

Payment purpose registry

A single payment can satisfy different business purposes: an order, a subscription, a listing promotion, or a BNPL instalment. The payments module activates the correct handler on verified success. This keeps payment verification independent of the domain that initiated it.

Purpose	Effect on verified success	Ledger posting
Order / Protected Checkout	Confirm payment, fund escrow, reserve stock already held, move order to funds safeguarded.	Debit provider receipts / credit escrow obligations (via partner mirror).
Subscription	Activate plan, grant featured credits, set period.	Debit provider receipts / credit subscription revenue.
Promotion	Activate listing promotion, set tier/featured window.	Debit provider receipts / credit promotion revenue.
BNPL instalment	Mark instalment paid, update plan status, notify.	Recorded against the financing partner's position.

6. Data architecture

54 tables in six domains. Money is `Decimal` with at most two decimal places and is serialised as a string; it never touches floats. The database enforces the invariants that matter most.

Domain	Core tables	Key guarantees
Identity & access	users, profiles, refresh_tokens, otp_codes, two_factor, admin roles/permissions, staff invitations, audit logs	Token rotation and reuse detection; append-only audit; granular RBAC.
Seller & catalog	seller_profiles, verifications, categories, products, product_images, favourites, reports	One seller profile per user; slug uniqueness; image ownership; verification tiers.
Messaging & negotiation	conversations, messages, read receipts, negotiations, offers	One live negotiation per buyer/product (partial unique index); stale-offer rejection.
Commerce	orders, order_items, payments, escrow_records, escrow_events, refunds, disputes, disputes_messages, reviews	Total = subtotal + delivery + protection; one active refund; append-only escrow events.
Growth	subscription_plans, subscriptions, promotions, platform_settings, ads/events, interest signals	Premium derived from an active period; promotion tier mirrors an active paid promotion.
Flex & ledger	credit_profiles, bank_connections, credit_assessments, guarantors, applications, agreements, instalments, ledger_accounts, ledger_transactions, ledger_entries	One running plan; guarantor cap; immutable assessment snapshots; balanced ledger at commit.

Ledger design

The ledger is double-entry and append-only. A deferred constraint trigger rejects any unbalanced transaction at commit. Database triggers block updates and deletes on ledger, escrow-event and audit tables.

Event	Debit	Credit
Escrow funded	`asset:escrow_partner_holding`	`liability:escrow_obligations`
Released to seller	`liability:escrow_obligations`	`liability:seller_payable:<seller>`, `revenue:platform_commission`, `revenue:protection_fees`
Refunded	`liability:escrow_obligations`	`asset:escrow_partner_holding`
Subscription	`asset:provider_receipts`	`revenue:subscriptions`
Promotion	`asset:provider_receipts`	`revenue:promotions`

7. API architecture

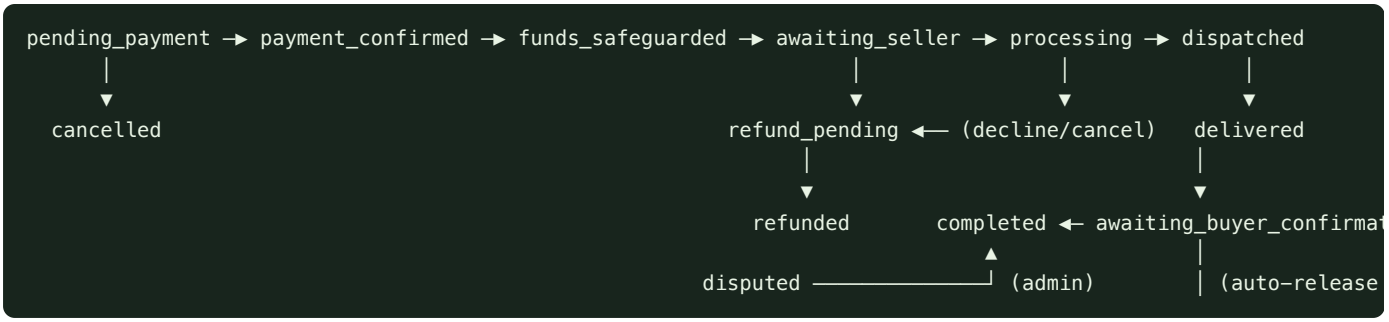
- **Style:** REST under `/api/v1`; JSON; 185 endpoints; OpenAPI in development/test only.
- **Auth:** Bearer access token (15 min) plus an httpOnly refresh cookie scoped to `/api/v1/auth`; CSRF header on cookie-authenticated endpoints.
- **Idempotency:** `Idempotency-Key` on checkout; unique `(provider, event_key)` on webhooks; unique ledger keys on settlement.
- **Errors:** consistent `{error:{code,message,details}}`; resource ownership failures return 404 to avoid existence leaks.
- **Rate limits:** per user/device and a coarse per-IP ceiling (20× by default) for carrier NAT; stricter per-action limits on login, OTP, offers, messages, uploads, checkout and Flex.
- **Webhooks:** `POST /api/v1/payments/webhooks/{provider}/webhook`; signatures verified (Paystack HMAC-SHA512, Flutterwave `verif-hash`); invalid signatures return 403; events de-duplicated.
- **Health:** `/health` for liveness, `/health/ready` checks the database and reports providers outside production.

8. Identity and access

Control	Implementation
Passwords	Argon2id; automatic re-hash on login; 8+ characters with letters and digits; timing-safe login for unknown emails.
Access tokens	Short-lived JWT (15 min) kept in memory; algorithm pinned to HS256/384/512; per-user token version revokes all on password/role/status change.
Refresh tokens	Opaque, hashed at rest, rotated on every use; reuse revokes the family; httpOnly, SameSite=Lax, Secure outside development.
OTP	6 digits, HMAC-peppered, 10-minute TTL, 5 attempts, 3 sends/10 min and 10/day per number.
2FA	TOTP with single-use recovery codes and replay protection; required for admins in production; staff cannot use Google sign-in.
Captcha	reCAPTCHA v2 or Turnstile, enforced before the password check for new devices (24 h) or after 3 wrong passwords.
Google sign-in	OIDC web flow; mock consent in development; hidden in production when credentials are absent.
RBAC	Granular admin permissions; only super admins change roles; every admin route has a permission dependency; audit log is append-only.
Proxy trust	`X-Forwarded-For` ignored unless the peer is in `TRUSTED_PROXY_CIDRS`; chain walked right-to-left.

9. Payments, escrow and ledger

Goods.ng does not hold funds. A licensed escrow/payment partner holds them; Goods.ng instructs release or refund based on the order state machine. The ledger mirrors the partner's position.



- **Verify before trust:** server-to-server verify with exact amount and currency; webhook signature; redirects never trusted.
- **No double settlement:** state machine + row locks + unique `escrow:<id>:settle` ledger key; a five-thread concurrency test asserts one settlement.
- **Server-side pricing:** totals from the listing or an accepted negotiation owned by the buyer; DB check constraint on the total.
- **Stock:** reserved atomically at checkout; restored on unpaid expiry or pre-dispatch refund; late payment honoured if stock remains, otherwise auto-refunded.
- **Refunds:** automatic pre-dispatch for cancellation/decline; dispute refunds need a second, different admin; one active refund per order.
- **Reconciliation:** `/admin/reconciliation` cross-checks payments, escrow states and ledger postings.

10. Goods Flex

Flex is a credit product, not a wallet. Before applying, the customer builds a credit profile: verified phone → BVN (Fernet-encrypted, keyed fingerprint to prevent reuse) → read-only bank connection → mandatory guarantor consent → optional workplace verification. Without a guarantor, the limit is zero and no application can start.

Step	Rule
Bank assessment	Six months of transactions; salary detection or 70% of median eligible inflow; deduct existing repayments; risk factor from volatility, missing months, betting share and failed debits.
Limit	`min(statement limit, verification-tier cap, repayment-history cap, lender maximum)`; zero without phone + BVN + bank + consented guarantor.
Guarantor	Sees borrower, relationship, amount and expiry; explicit consent; consent time, IP and BVN reference recorded; amount caps the limit; at most 3 active guarantees per person; withdrawal drops the limit to zero.
Decision	Partner decision stored with reference; offer terms stored verbatim; buyer explicitly accepts.
One plan	Only one of approved/accepted/active/defaulted at a time, enforced in the service under a row lock and by a partial unique index.
Funds	Down payment via the payment layer; partner disburses; order funded through the same escrow path; refund cancels the agreement.
Repayment	Instalments via the payment layer; worker marks overdue and reminds; defaulted plans can still be paid off.

11. Search and discovery

- **Engine:** PostgreSQL full-text search; generated weighted `tsvector` (title A, brand/model B, description C) with a GIN index and prefix matching (`iph` matches iPhone).
- **Ranking:** featured first, then premium sellers, then text rank, then recency.
- **Filters/facets:** state, category, price, condition, brand, BNPL, delivery, verification; counts computed for the current result set.
- **Sponsored:** premium sellers and Boost Premium listings enter a quality-gated sponsored pool; TOP promotes a single listing.
- **Seam:** `products.service.search()` is the single place to swap in a dedicated engine later.
- **Recommendations:** interest signals and a "for you" feed; purge job keeps signal tables bounded.

12. Messaging and negotiation

Area	Behaviour
Conversations	Product-linked; one thread per buyer/seller/product context; read receipts and unread counts; 5-second polling while visible, paused when hidden.
Safety	Direct-contact warning gate; message reporting; rate limits; HTML stripped.
Negotiation	One live negotiation per buyer/product; turn-taking; counter direction; minimum offer 10% of list; 48-hour offer expiry; 24-hour checkout after acceptance; 10-round cap.
Concurrency	Every response locks the negotiation row and must name the pending offer; stale/double clicks get `409 stale_offer`.
Price integrity	The public listing price never changes; the accepted offer becomes the checkout price for that buyer only.

13. Frontend architecture

- **Public pages** (home, search, category, item, store, info) are server-rendered with streaming and PPR; static shells are prerendered; JSON-LD, canonical URLs, OG metadata, sitemap and robots are included.
- **Dashboards** (account, orders, messages, seller, admin) are client components on a central API client: in-memory access token, silent refresh through the httpOnly cookie with a CSRF header, one retry on 401.
- **Performance:** system fonts, backend WebP thumbnails (400 px) and full images (1280 px), lazy loading, skeletons, Redis-cached feeds, bottom navigation on phones.
- **Admin host:** `frontend/src/proxy.ts` serves the console on `admin.goods.ng`, redirects `/admin/...`, and adds `X-Robots-Tag: noindex`. Sessions are per host.
- **Resilience:** no client-side price trust, no direct provider secrets, and no analytics scripts before marketing consent.

14. Media pipeline

1. Input capped at `MAX_UPLOAD_MB` (8 MB) and 40 MP (decompression-bomb guard).
2. Decoded with Pillow; extension and MIME are not trusted.
3. Re-encoded to WebP: full at 1280 px, thumbnail at 400 px; EXIF/GPS and embedded payloads stripped.
4. Listing photos watermarked with the seller business name and Goods.ng.
5. Stored under random keys; public media to S3 + CDN; KYC documents to the private prefix with server-side encryption and short-lived signed URLs.
6. Per-seller cap `MAX_IMAGES_PER_SELLER` (200); soft-deleted listings purged after 30 days.

15. Background processing

Job	Interval	Purpose
`outbox`	10 s	Drain queued email/SMS notifications.
`expire_unpaid_orders`	60 s	Expire unpaid orders, restore stock, re-check late payments with the provider.
`expire_negotiations`	60 s	Expire stale offers and accepted-offer checkout windows.
`auto_release_escrow`	300 s	Release funds after the buyer-confirmation window.
`expire_subscriptions`	300 s	Expire plans, clear promoted tiers and features.
`refresh_promotion_bumps`	3600 s	Refresh Boost Premium top placement.
`bnpl_reminders`	3600 s	Mark overdue instalments and send reminders.
`meta_capi_purchases`	300 s	Send consented purchase events to Meta Conversions API.
`purge_interest_signals`	86400 s	Keep recommendation signals bounded.
`purge_deleted_listings`	86400 s	Hard-delete listings and media after retention.

16. Caching and performance

- **Cache:** Redis with an in-memory development fallback; anonymous search cached for 15 s; public feeds cached; single-flight locks prevent stampedes.
- **Rate limiting:** per user/device and a coarse per-IP ceiling; hot paths use constant-time checks.
- **Database:** 15 s statement timeout, 60 s idle-in-transaction timeout, pooled connections, indexes for search and browse.
- **Measurement:** k6 realistic mix; single dev worker plateaus at ~182 req/s. Add workers/replicas and PgBouncer before that becomes the ceiling.

Scenario	Throughput	p95	Errors
Home feed smoke (20 concurrent)	~300 req/s	~220 ms	0
Search smoke (20 concurrent)	~240 req/s	~125 ms	0
k6 mix, 120 VUs	—	41 ms	0
k6 mix, 400 VUs	182 req/s	553 ms	0
k6 mix, 800 VUs	~182 req/s (plateau)	~1.9 s	0

17. Security architecture

Area	Control
Transport & headers	TLS everywhere; HSTS in staging/production; CSP, nosniff, frame DENY, referrer policy, Permissions-Policy, COOP; `no-store` on auth/admin responses; no Server header in production.
Secrets	Secret manager only; production boot rejects dev defaults, short JWT secrets, http origins, missing Redis, local storage, mock financial providers, missing proxy CIDRs and admin 2FA off.
Input/output	Pydantic validation; HTML stripped with nh3; React escaping; JSON-LD escaped; user URLs restricted to http(s); parameterised queries only.
Files	Decode-and-re-encode; random keys; private KYC prefix; signed URLs; malware scanning still outstanding.
Financial	Append-only ledger/escrow/audit; balance check at commit; row locks; idempotency; four-eyes refunds; reconciliation.
Privacy	KYC identifiers encrypted; masked in API; admin reveal audited; marketing cookies gated; NDPA workflows partially outstanding.
Audit	Every staff sign-in/out/failure, 2FA failure, admin action, permission refusal and console page view; CSV export; immutable rows.
Outstanding	Malware scanning, licensed KYC provider, external pen test, DPO/data-subject workflow, NDPA record of processing.

18. Observability and operations

- **Logs:** structured JSON on stdout with `request_id` on every line and in the `x-request-id` response header; ship to a log platform.
- **Health:** `/health` (liveness) and `/health/ready` (DB, cache, providers outside production).
- **Error tracking:** Sentry SDK for FastAPI and Next.js is **not wired yet**; add DSNs via secrets and extend CSP allowlists.
- **Alerting:** 5xx rate, p95 latency, readiness failures, worker job failures, payment amount mismatch, invalid webhook signatures, non-empty reconciliation, overdue BNPL growth, guarantor limit spikes, provider errors, bank-connection drop.
- **Backups:** managed PITR 7–35 days plus encrypted daily logical dumps to a separate account/region; quarterly restore drill into staging and ledger reconciliation.
- **Rollback:** redeploy the previous immutable image tag; prefer forward database fixes; never blindly downgrade financial tables.

19. Integration architecture and go-live readiness

This section is the architecture-level view of the integration gaps. Appendix A is the detailed checklist with owners and acceptance criteria.

Integration	Interface / state today	Go-live	What remains
Escrow partner	`EscrowProvider`; mock only	BLOCKER	Licensed contract; implement create_hold/release/refund; sandbox certification; payout API; webhooks; reconciliation.
Lending partner (Flex)	`FinancingProvider`; mock only	BLOCKER	Licensed lender; decision/disbursement/repayment APIs; terms; FCCPC review; collections.
Payments	Paystack and Flutterwave adapters; mock active	REQUIRED	Sandbox → live certification; webhook secrets; settlement account; refunds; reconciliation.
Open banking	Mono adapter; mock active	REQUIRED	Mono sandbox test; Connect widget keys; production approval; connection webhooks.
KYC / BVN	`IdentityProvider`; mock only	REQUIRED	Licensed provider adapter (Dojah/Prembly/Smile ID/Youverify); consent; encryption; webhook; manual fallback.
SMS	Termii adapter; console active	REQUIRED	Sender ID approval; templates; delivery reports; DND; sandbox OTP test.
Email	Postmark/SMTP adapters; console active	REQUIRED	Domain + SPF/DKIM/DMARC; templates; bounce/complaint handling; sandbox test.
Object storage / CDN	Local + S3 adapter; local active	REQUIRED	Bucket, IAM, CORS, CDN, lifecycle, versioning, private KYC prefix, signed URLs.
Captcha	reCAPTCHA/Turnstile; mock active	REQUIRED	Production keys and domains; CSP already allows providers.
Google OAuth	Implemented; mock in dev	CONFIG	OAuth client, consent screen, redirect URIs, publish app.
Redis	Optional in dev; required in prod	BLOCKER	Managed instance, private network, TLS, monitoring.
Sentry / observability	Not wired	REQUIRED	SDKs, DSNs, release tracking, alerts, on-call, log platform, uptime.
Deployment platform	Dockerfiles/Compose; not deployed	BLOCKER	Vercel, container platform, managed Postgres PITR, PgBouncer, DNS/SSL.
Seller payouts	Ledger only (`seller_payable`)	REQUIRED	Partner payout API; bank account verification; payout scheduling; UI; reconciliation.
Logistics / tracking	Manual tracking fields	RECOMMENDED	At least one carrier; labels; tracking webhooks; delivery status mapping.
Malware scanning	Not implemented	REQUIRED	ClamAV/cloud scanner on upload; quarantine; KYC document handling.
Google Merchant / Ads	Feed endpoint implemented	GROWTH	Merchant Center, domain claim, scheduled fetch, Ads link, campaign.
Meta Pixel / Catalog / CAPI	Pixel events, feed and CAPI implemented	GROWTH	Business Manager, catalog, domain verification, pixel ID, CAPI token, campaign.
GA4 / Google Ads tags	Admin settings + consent gating	GROWTH	Property/tag IDs; conversion labels; consent-mode check.
Push notifications	Not implemented	OPTIONAL	FCM/APNs, device tokens, preferences; needed for native apps.

Integration	Interface / state today	Go-live	What remains
WhatsApp Business API	Deep links only	OPTIONAL	Cloud API, templates, opt-in, handoff to human agents.
Dedicated search	Postgres FTS	SCALE	OpenSearch/Meilisearch behind `service.search()`.
WebSockets / SSE	5-second polling	SCALE	Replace polling if load or latency demands.
Fraud / risk scoring	Rules + verification only	OPTIONAL	Vendor scoring, device fingerprint, velocity rules.

20. Scalability plan

1. **Horizontal API:** add replicas; keep the app stateless; move sessions/tokens as already designed.
2. **Database:** PgBouncer transaction pooling; read replicas for browse/search; partition large event tables if needed; then shard only as a last resort.
3. **Cache:** increase Redis capacity; add per-entity cache invalidation; cache anonymous feeds and facets.
4. **Search:** move to a dedicated engine when Postgres FTS p95 or query complexity becomes the bottleneck.
5. **Messaging:** WebSockets/SSE or a managed pub/sub when polling at scale becomes expensive.
6. **Media:** CDN offload, responsive variants, signed URLs; keep origin egress low.
7. **Worker:** more replicas are safe; split jobs by queue if contention appears.

21. Architecture decisions

ADR	Decision	Rationale
001	Modular monolith, not microservices	Small team, shared transactions, fast iteration; module seams allow later extraction.
002	PostgreSQL as the only source of truth	Strong constraints, transactions, FTS and operational simplicity; ledger integrity is paramount.
003	Provider interfaces for regulated services	Partners change; the domain must not. Production refuses mocks.
004	Same-origin API via Next.js rewrite	First-party refresh cookie, no CORS, simpler CSP, better mobile reliability.
005	Double-entry ledger with DB triggers	Money correctness cannot depend on application discipline alone.
006	Verify-before-trust payments	Webhooks and redirects are signals, not proof.
007	Premium is derived, not stored	An active subscription period is the only truth; clients cannot self-grant.
008	Per-listing promotions separate from subscriptions	Lowers the barrier: a seller can buy visibility for one product without a plan.
009	Postgres FTS first	Sufficient at launch; one seam to replace later.
010	Polling messaging first	Simpler operations; WebSockets can replace without API changes.

22. Technical risks

Risk	Mitigation
Escrow/lending adapter surprises in sandbox	Contract-first adapter tests; certify in staging before production; keep mock for regression tests.
Webhook storms or missed events	Idempotency, de-duplication, buyer-return verification, expiry job re-check, alerting.
Database connection exhaustion	Pool math, PgBouncer, load tests, connection alerts.
Large-image abuse	Size and pixel caps, re-encode, per-seller quota, malware scan.
KYC data breach	Private storage, encryption, signed URLs, least privilege, audit, pen test.
Provider outage	Retries with backoff, status page, manual reconciliation, graceful buyer messaging.

Risk	Mitigation
Search relevance drift	Ranking tests, stable weights, search success metric.
Polling load at scale	Visibility-aware polling, backoff, then WebSockets/SSE.

A. Integration go-live checklist

Status vocabulary: LIVE implemented and tested against mocks/local; CONFIG code exists but production credentials/dashboard work remains; MOCK only a mock adapter exists; NOT BUILT no implementation. "Blocker" means the item must be complete before the first live transaction or before the production process will boot.

#	Integration	Why it matters	Current state	What remains	Blocker	Owner
1	**Licensed escrow / payment partner**	Holds buyer funds for Protected Checkout; Goods.ng must not hold funds.	MOCK EscrowProvider only	Contract; implement hold/release/refund; sandbox certification; settlement/payout API; webhooks; reconciliation.	Yes	Finance + Eng
2	**Licensed BNPL lender**	Provides the credit decision, disbursement and collections for Goods Flex.	MOCK FinancingProvider only	Contract; decision/disbursement/repayment adapter; real terms; FCCPC review; collections process.	Yes	Risk + Legal
3	**Payment provider** — Paystack or Flutterwave	Collects card/transfer/bank payments and verifies them server-side.	CONFIG both adapters written, untested	Sandbox account; certify initialize/verify/refund; webhook secret and URL; settlement account; reconciliation; choose primary/backup.	Yes	Finance + Eng
4	**KYC / BVN-NIN provider**	Verifies seller identity and Flex borrowers; supports the "Verified" badge at scale.	MOCK IdentityProvider only	Choose Dojah/Prembly/Smile ID/Youverify; adapter; consent; encryption; webhook; manual review fallback; NDPA terms.	Yes	Compliance + Eng
5	**Open banking** — Mono	Read-only bank statements for the Flex affordability assessment.	CONFIG Mono adapter, untested	Mono sandbox; Connect widget keys; production approval; connection webhooks; error monitoring; consent copy.	No*	Risk + Eng
6	**SMS** — Termii	OTP, guarantor consent links and transactional alerts.	CONFIG adapter, console active	Sender ID approval; templates; DND handling; delivery reports; sandbox OTP proof; spend limits.	Yes	Eng + Ops
7	**Email** — Postmark or SMTP	Verification, receipts, dispute and Flex notifications.	CONFIG adapters, console active	Domain verification; SPF/DKIM/DMARC; templates; bounce/complaint webhooks; sandbox test.	Yes	Eng + Ops
8	**Object storage + CDN**	Durable public listing media and private KYC documents.	CONFIG S3 adapter, local active	Bucket; IAM least privilege; CORS; CDN; versioning; lifecycle; private prefix; signed URLs; encryption.	Yes	Platform
9	**Managed Redis**	Shared cache, rate limits and single-flight locks.	CONFIG optional in dev	Provision with TLS/private network; monitoring; eviction policy; boot check in production.	Yes	Platform
10	**Captcha** — reCAPTCHA v2 or Turnstile	Bot protection on sign-in for new devices and after failures.	CONFIG providers implemented, mock active	Production keys; allowed domains; CSP already allows; verify server-side in staging.	Yes	Eng

#	Integration	Why it matters	Current state	What remains	Blocker	Owner
11	**Sentry / error tracking**	Detect and triage production errors in API and frontend.	NOT BUILT	Add FastAPI + Next.js SDKs; DSNs in secrets; release tags; source maps; alert routing; CSP updates.	Yes	Platform
12	**Deployment platforms**	Run the API, worker, frontend, database and cache in production.	NOT BUILT Dockerfiles/Compose only	Vercel project; container platform; ≥2 API instances; worker; managed Postgres PITR; PgBouncer; DNS/SSL.	Yes	Platform
13	**Seller payouts / disbursement**	Move released seller balances out of the escrow partner to seller bank accounts.	NOT BUILT ledger only	Partner payout API; bank account verification/name lookup; payout scheduling; retry/failed handling; UI; reconciliation.	Yes	Finance + Eng
14	**Malware scanning**	Scan listing images and KYC documents before storage/access.	NOT BUILT	ClamAV or cloud scanner; quarantine bucket; scan status; admin review; alerting.	Yes	Security
15	**Logistics / carrier**	Labels, pickup and tracking events for dispatched orders.	NOT BUILT manual tracking fields	At least one carrier (GIGL, DHL, Sendbox, etc.); label API; tracking webhooks; status mapping; failed-delivery flow.	No	Ops + Eng
16	**Google Merchant Center + Ads**	Product listing ads and Shopping/Performance Max campaigns.	CONFIG feed endpoint	Merchant Center; claim goods.ng; Nigeria/NGN; scheduled fetch; link Ads; campaign; feed quality checks.	No	Growth
17	**Meta Pixel / Catalog / CAPI**	Catalog ads and consented server-side purchase events.	CONFIG pixel, feed, CAPI code	Business Manager; catalog data source; domain verification; pixel ID; CAPI token; campaign; event dedup check.	No	Growth
18	**GA4 / Google Ads tags**	Acquisition and conversion measurement.	CONFIG settings + consent gate	Property/tag IDs; conversion labels; consent mode; verify no scripts before consent.	No	Growth
19	**Google OAuth**	Optional social sign-in.	CONFIG implemented, mock in dev	OAuth web client; consent screen; redirect URIs; publish/verify app; privacy/terms links.	No	Eng
20	**Push notifications**	Re-engage buyers/sellers; required for future native apps.	NOT BUILT	FCM/APNs; device tokens; preferences; deep links; delivery analytics.	No	Eng
21	**WhatsApp Business API**	Structured messaging and template notifications.	NOT BUILT deep links only	Cloud API; number; templates; opt-in; webhook; human handoff; 24-hour window handling.	No	Growth + Eng
22	**Dedicated search**	Search relevance and latency at scale.	NOT BUILT Postgres FTS	OpenSearch/Meilisearch; index pipeline; ranking parity; A/B; swap behind `service.search()`.	No	Eng
23	**WebSockets / SSE**	Lower-latency messaging if polling becomes costly.	NOT BUILT 5 s polling	Gateway; auth; fan-out; presence; reconnect; load test; mobile background behaviour.	No	Eng
24	**Fraud / risk scoring**	Detect abusive signups, listings and payments.	NOT BUILT rules only	Vendor or in-house scoring; device fingerprint; velocity rules; manual review queue; model governance.	No	Trust & Safety

#	Integration	Why it matters	Current state	What remains	Blocker	Owner
25	**NDPA / DPO tooling**	Data-subject export/delete, retention and processing records.	PARTIAL copy only	DSR workflow; retention jobs; record of processing; DPO appointment; processor agreements; breach playbook.	Yes	Compliance

* Mono is not a production boot blocker, but Goods Flex cannot go live without it (or another open-banking provider) because the credit limit depends on a real bank assessment. If Flex is feature-flagged off at launch, Mono can follow.

Top five blockers, in order

1. Licensed escrow/payment partner (Protected Checkout cannot launch without it). **2.** Licensed lender (Goods Flex cannot launch without it). **3.** Payment provider certification (no live money). **4.** Production deployment + managed infrastructure (nothing is live). **5.** KYC + SMS/email + captcha (security and account recovery).

B. Environment variables

Variable	Purpose / production requirement
`ENVIRONMENT`	development | test | staging | production. Controls boot guards and dev routes.
`DATABASE_URL`	PostgreSQL DSN; must use sslmode=require/verify-full in production.
`DB_POOL_SIZE / DB_MAX_OVERFLOW / DB_POOL_TIMEOUT`	Per-process pool; size against Postgres max_connections or PgBouncer.
`REDIS_URL`	Required in production; shared cache, rate limits, single-flight.
`JWT_SECRET / OTP_SECRET / DATA_ENCRYPTION_KEY`	Unique per environment; production rejects dev defaults; back up the Fernet key.
`FRONTEND_URL / CORS_ORIGINS / ADMIN_URL`	Explicit https origins; admin subdomain used in staff invites.
`TRUSTED_PROXY_CIDRS`	Load balancer CIDRs; required in production for correct client IPs.
`ADMIN_2FA_REQUIRED`	Must be true in production; admins enable TOTP before using the admin API.
`PAYMENT_PROVIDER`	mock | paystack | flutterwave; mock refused in production.
`ESCROW_PROVIDER / BNPL_PROVIDER`	Mock only today; production refuses mock; real adapters required.
`BANK_DATA_PROVIDER / MONO_*`	mock | mono; Mono keys server-side only; production refuses mock.
`PAYSTACK_SECRET_KEY / FLUTTERWAVE_*`	Provider credentials and webhook hash.
`SMS_PROVIDER / TERMII_*`	console | termii; production refuses console.
`EMAIL_PROVIDER / SMTP_* / POSTMARK_*`	console | smtp | postmark; production refuses console.
`STORAGE_BACKEND / S3_* / MEDIA_BASE_URL`	local | s3; production requires s3 + bucket; CDN URL for media.
`CAPTCHA_PROVIDER / *_SITE_KEY / *_SECRET_KEY`	off | mock | recaptcha | turnstile; production refuses mock/off.
`GOOGLE_CLIENT_ID / SECRET`	Google OAuth; hidden in production when absent.
`META_CAPI_ACCESS_TOKEN / META_GRAPH_VERSION`	Server-side purchase events; token never in DB/client.

Variable	Purpose / production requirement
FEED_TOKEN	Optional protection for Google/Meta feeds.
PLATFORM_COMMISSION_RATE / PROTECTION_FEE_RATE / PROTECTION_FEE_CAP	Marketplace economics; admin-overridable.
AUTO_RELEASE_DAYS	Buyer-confirmation window before escrow release (default 3).
RATE_LIMIT_PER_MINUTE / RATE_LIMIT_IP_MULTIPLIER	Per-identity and coarse per-IP ceilings.
MAX_UPLOAD_MB / MAX_IMAGES_PER_SELLER / DELETED_LISTING_RETENTION_DAYS	Media abuse and retention controls.
RATE_LIMIT_MULTIPLIER	Dev/test only; must be 1 outside development.

C. Glossary

Term	Meaning
Protected Checkout	Goods.ng checkout where a licensed partner holds the buyer's funds until delivery is confirmed or the dispute process resolves.
Escrow partner	A licensed third party that holds and releases funds. Goods.ng does not hold funds.
Goods Flex	Buy-now-pay-later product with an affordability assessment, mandatory guarantor and instalments.
Guarantor	A person who consents to cover a Flex borrower's obligation up to a stated, capped amount.
Premium seller	Derived status from an active paid subscription; unlocks contact buttons, analytics, banner and ranking.
Promotion	A one-off paid visibility package for a single listing: TOP 7, TOP 30 or Boost Premium 28.
Boost Premium	Promotion tier that makes a listing eligible for the sponsored pool while it meets quality gates.
Ledger	Double-entry, append-only record of money movements; balanced at commit by a deferred trigger.
Seller payable	Ledger liability representing money owed to a seller after release, before disbursement.
Verify-before-trust	Server-to-server payment verification with exact amount/currency; webhooks are only triggers.
Idempotency	The property that retrying the same request has no additional effect.
Four-eyes	A second, different admin must approve a dispute refund.
NDPA	Nigeria Data Protection Act 2023.
FCCPC	Federal Competition and Consumer Protection Commission; regulates digital lending conduct.
CBN	Central Bank of Nigeria; licenses payment/escrow institutions.
PPR	Partial Prerendering (Next.js); static shell with streamed dynamic data.
FTS	PostgreSQL full-text search.
PgBouncer	Connection pooler used in front of Postgres at scale.
PITR	Point-in-time recovery for the database.

Prepared from the Goods.ng repository and verified build state on 8 November 2026. This document is a planning and acceptance baseline, not legal advice. Licensed partner terms, regulatory interpretations and go-live dates must be confirmed by counsel and the relevant providers.